

Računarska grafika

Teksture

Vesna Marinković

Kako modelovati antičku piramidu?



Kako modelovati antičku piramidu?



- Možemo napraviti **veoma detaljnu geometriju objekta**
– veliki broj malih trouglova

Kako modelovati antičku piramidu?



- Možemo napraviti **veoma detaljnu geometriju objekta**
 - veliki broj malih trouglova
 - Nedostaci: vremenski i memorijski zahtevno generisanje, dugo se renderuje

Kako modelovati antičku piramidu?



- Možemo napraviti **veoma detaljnu geometriju objekta**
 - veliki broj malih trouglova
 - Nedostaci: vremenski i memorijski zahtevno generisanje, dugo se renderuje
 - Prednost: realistično modelovanje osvetljenja

Kako modelovati antičku piramidu?



- Možemo napraviti **veoma detaljnu geometriju objekta**
 - veliki broj malih trouglova
 - Nedostaci: vremenski i memorijski zahtevno generisanje, dugo se renderuje
 - Prednost: realistično modelovanje osvetljenja
- Da li možemo da proizvedemo složene vizualne elemente **bez menjanja geometrije** objekta?

Motivacioni primer: modelovanje Zemlje



- Zemlju, gledano iz svemira, možemo aproksimirati sferom

Motivacioni primer: modelovanje Zemlje



- Zemlju, gledano iz svemira, možemo aproksimirati sferom
- Ne želimo da modelujemo geometrijske detalje koji odgovaraju planinama, rekama, obalama, ...

Preuzeto sa slajdova kursa Univerziteta Brown



Motivacioni primer: modelovanje Zemlje

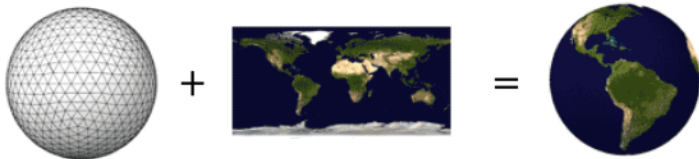


- Imamo na raspolaganju mapu Zemlje: sliku kako izgleda površina Zemlje kada se razvije u ravan

Motivacioni primer: modelovanje Zemlje



- Imamo na raspolaganju mapu Zemlje: sliku kako izgleda površina Zemlje kada se razvije u ravan
- Oko sfere možemo omotati sliku mape i dobiti željeni vizualni efekat

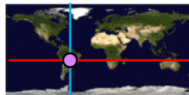
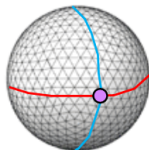


Motivacioni primer: modelovanje Zemlje



- Kako za proizvoljnu tačku na sferi odrediti kojoj tački sa mape odgovara?

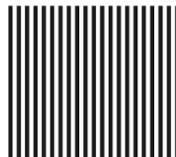
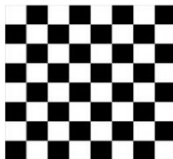
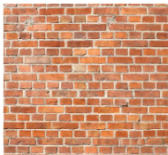
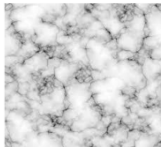
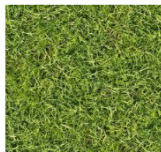
Motivacioni primer: modelovanje Zemlje



- Svaka tačka sa površine Zemlje ima svoju geografsku dužinu i širinu
- Koristimo ih za računanje kojoj tački sa mape odgovara ta tačka

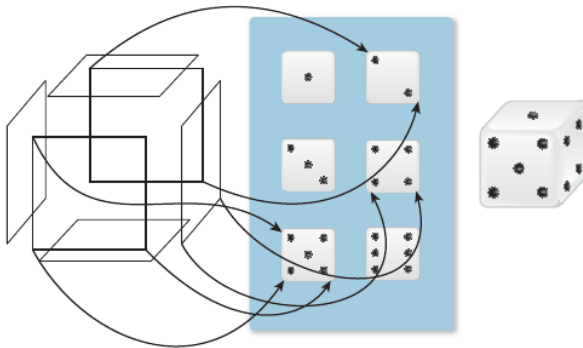
Teksture

Texture
(images)



Preslikavanje tekstura

- **Preslikavanje (mapiranje) tekstura** – svakoj tački date površi (domen) dodeljujemo odgovarajuću vrednost sa slike teksture (kodomen)
- Katmul, 1974.

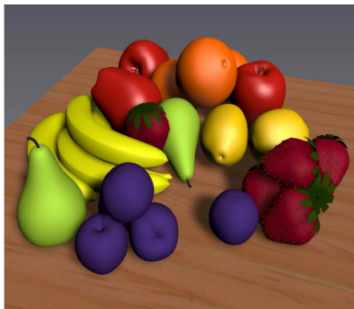


Zašto su nam potrebne teksture?

- Teksture su pogodne za prikaz:
 - grubih materijala
 - složenih scena
 - objekata koji imaju repetitivnu strukturu
- Iako grafičke kartice mogu da renderuju milione poligona u sekundi, to nije dovoljno za pojave kao što su oblaci, trava, zemljište, koža

Zašto su nam potrebne teksture?

- Teksture popravljaju realističnost prikaza



Prednosti i nedostaci preslikavanja tekstura

- Prednosti:

- mogu se jednom sačuvati i koristiti veći broj puta
- renderuju se jako brzo
- posebno korisne za daleke objekte poput reljefa, neba

Prednosti i nedostaci preslikavanja tekstura

- Prednosti:

- mogu se jednom sačuvati i koristiti veći broj puta
- renderuju se jako brzo
- posebno korisne za daleke objekte poput reljefa, neba

- Nedostaci:

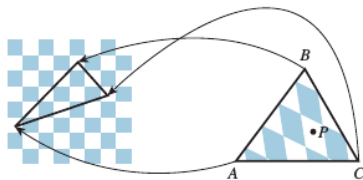
- gruba aproksimacija stvarnog života
- površi sa nalepljenim teksturama i dalje deluju glatko jer njihova geometrija ostaje nepomenjena

Teksturane koordinate

- Mapa teksture je najčešće slika pravougaonog oblika
- Na tačke sa slike teksture referišemo korišćenjem **teksturnih koordinata** u i v čije su vrednosti iz intervala $[0, 1]$

Teksturne koordinate

- Mapa tekstone je najčešće slika pravougaonog oblika
- Na tačke sa slike tekstone referišemo korišćenjem **teksturnih koordinata** u i v čije su vrednosti iz intervala $[0, 1]$
- Temenima trougla pridružuju se tačke sa mape tekstone
- Boja tačke P unutar trougla određuje se na osnovu koordinata njegovih temena



Preslikavanje tekstura

- Preslikavanje tekstura se sastoji iz dve etape:

Preslikavanje tekstura

- Preslikavanje tekstura se sastoji iz dve etape:
 - ① **Parametrizacija površi:** „odmotavanje“ 3D površi u ravan
 - formalno, računanje funkcije $f(x, y, z) = (u, v)$ koja opisuje površ
 - izračunavanje vrednosti koordinata u i v za svaku tačku površi

Preslikavanje tekstura

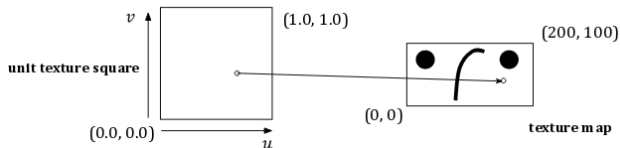
- Preslikavanje tekstura se sastoji iz dve etape:
 - 1 **Parametrizacija površi:** „odmotavanje“ 3D površi u ravan
 - formalno, računanje funkcije $f(x, y, z) = (u, v)$ koja opisuje površ
 - izračunavanje vrednosti koordinata u i v za svaku tačku površi
 - 2 **Teksturisanje:** dodeljivanje vrednosti svakoj tački (u, v) ravni
 - vrednosti mogu biti boje, drugi vizualni parametri (npr. eksponent spekularne refleksije), čak i vektori normale površi

Teksturisanje

- Teksturisanje je često jednostavnija etapa u preslikavanju tekstura
- Razlikujemo dve vrste teksturisanja:
 - **teksturisanje slikom** – koordinate (u, v) se koriste za indeksiranje pozicije na slici koja sadrži teksturu
 - **proceduralno teksturisanje** – koordinate (u, v) su ulazni parametri funkcije kojom se računa vrednost teksture

Teksturisanje slikom

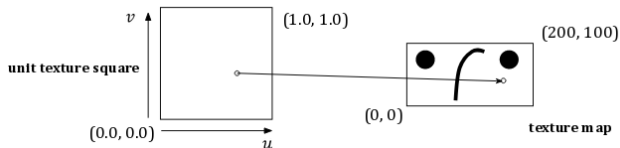
- Preslikavanje tačke (u, v) jediničnog kvadrata na teksturu širine w i visine h
 - Odgovarajuća tačka sa mape teksture je proporcionalna po svakoj osi



- U ovom primeru: $(0.0, 0.0) \rightarrow (0, 0)$, $(1.0, 1.0) \rightarrow (200, 100)$,
 $(0.7, 0.45) \rightarrow (140, 45)$
- Kada dobijemo koordinate sa slike teksture, dovoljno je pročitati vrednost teksture na datim koordinatama

Teksturisanje slikom

- Preslikavanje tačke (u, v) jediničnog kvadrata na teksturu širine w i visine h
 - Odgovarajuća tačka sa mape teksture je proporcionalna po svakoj osi



- U ovom primeru: $(0.0, 0.0) \rightarrow (0, 0)$, $(1.0, 1.0) \rightarrow (200, 100)$, $(0.7, 0.45) \rightarrow (140, 45)$
- Kada dobijemo koordinate sa slike teksture, dovoljno je pročitati vrednost teksture na datim koordinatama
- Dobijene koordinate teksture nisu uvek celobrojne jer je prostor u, v koordinata neprekidan

Nanošenje tekstura

- **Skaliranje** – koristi se kao zamena za jako složeni model, ciljna površina se pokriva skaliranjem slike teksture

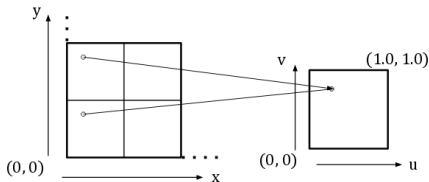


- **Popločavanje** – koristi se za simulaciju materijala konzistentnog izgleda, ciljna površina se pokriva ponavljanjem slike teksture



Popločavanje tekstura: implementacija

- Razmotrimo popločavanje proizvoljno velikog pravougaonika pravougaonicima dimenzije (w, h)

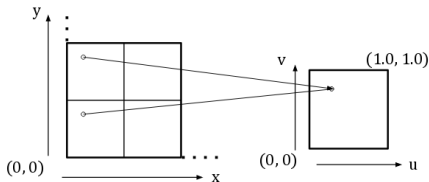


- Za tačku sa koordinatama (x, y) sa velikog pravougaonika (u, v) koordinate teksture mogu se izračunati po formuli:

$$(u, v) = \left(\frac{x \% w}{w}, \frac{y \% h}{h} \right)$$

Popločavanje tekstura: implementacija

- Razmotrimo popločavanje proizvoljno velikog pravougaonika pravougaonicima dimenzije (w, h)



- Za tačku sa koordinatama (x, y) sa velikog pravougaonika (u, v) koordinate teksture mogu se izračunati po formuli:

$$(u, v) = \left(\frac{x \% w}{w}, \frac{y \% h}{h} \right)$$

- Operacijom $\%$ se vrednosti (x, y) svode na opseg $[0, w)$, odnosno $[0, h)$, a deljenjem sa w , odnosno h na opseg $[0, 1)$

Parametrizacija površi

- Prva komponenta procesa preslikavanja tekstura
- Treba odrediti funkciju $f(x, y, z) = (u, v)$ kojom se opisuje površ
- Ovde leži najveća težina preslikavanja tekstura

Parametrizacija površi

- Prva komponenta procesa preslikavanja tekstura
- Treba odrediti funkciju $f(x, y, z) = (u, v)$ kojom se opisuje površ
- Ovde leži najveća težina preslikavanja tekstura
- Nalaženje **dobre parametrizacije**, tj. one koja neće mnogo izobličiti teksturu kada se primeni na proizvoljnu površ je i dalje otvoren istraživački problem

Parametrizacija površi

- Prva komponenta procesa preslikavanja tekstura
- Treba odrediti funkciju $f(x, y, z) = (u, v)$ kojom se opisuje površ
- Ovde leži najveća težina preslikavanja tekstura
- Nalaženje **dobre parametrizacije**, tj. one koja neće mnogo izobličiti teksturu kada se primeni na proizvoljnu površ je i dalje otvoren istraživački problem
- Mi ćemo razmatrati dva slučaja:
 - jednostavan slučaj – primitiva koju je lako parametrizovati
 - teži slučaj – parametrizacija proizvoljne mreže trouglova

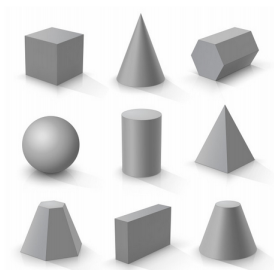
Parametarsko definisanje površi

- Određene 3D primitive se prirodno definišu parametarskom funkcijom $g(u, v) = (x, y, z)$: valjak, kupa, sfera, ...
- Potrebno je odrediti tri funkcije:

$$x = x(u, v)$$

$$y = y(u, v)$$

$$z = z(u, v)$$



- Podsetimo se kako se parametarski mogu zadati valjak i sfera

Parametarsko zadavanje valjka

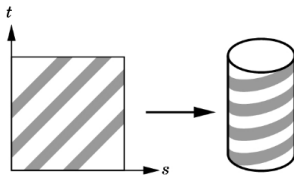
- Preslikavanje pravougaonika parametrizovanog sa u i v na omotač valjka poluprečnika r i visine h čiji je centar osnove u koordinatnom početku može se zadati kao:

$$x = r \cdot \cos 2\pi u$$

$$y = h \cdot v$$

$$z = r \cdot \sin 2\pi u$$

- Sa $\theta = 2\pi u$ zadat je ugao koji projekcija tačke P na ravan Oxz zaklapa sa x -osom
- Koordinatom y zadaje se visina tačke P



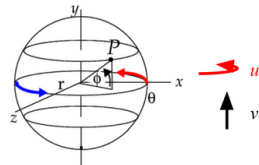
Parametarsko zadavanje sfere

- Preslikavanje pravougaonika parametrizovanog sa u i v na sferu poluprečnika r sa centrom u koordinatnom početku može se zadati kao:

$$x = r \cdot \sin 2\pi u \cdot \cos \pi v$$

$$y = r \cdot \cos 2\pi u$$

$$z = r \cdot \sin 2\pi u \cdot \sin \pi v$$



- Sa $\phi = \pi v$ zadat je ugao koji tačka P zaklapa sa svojom projekcijom na ravan Oxz , a sa $\theta = 2\pi u$ ugao koji projekcija tačke P zaklapa sa x -osom
- Pri ovom preslikavanju javlja se distorzija na polovima

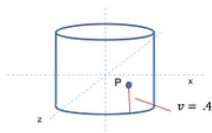
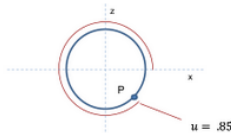
Preslikavanje teksture na valjak

- Nama je potreban obrnuti smer: da za datu tačku sa valjka/sfere odredimo kojoj tački teksture odgovara
- Potrebno je odrediti preslikavanje oblika:

$$u = u(x, y, z)$$

$$v = v(x, y, z)$$

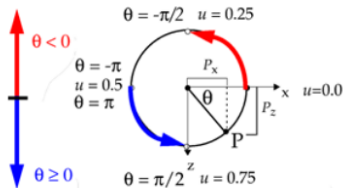
- Razmotrimo tačku P koja pripada omotaču jediničnog valjka sa osnovom u ravni $z = 0$ i središtem osnove u koordinatnom početku
 - koristimo poziciju tačke u odnosu na obim kruga da izračunamo koordinatu u
 - koristimo visinu tačke da izračunamo koordinatu v



Preslikavanje teksture na valjak (2)

- Koordinata v jednaka je vrednosti y koordinate
- Treba preslikati sve tačke sa kruga u opseg $[0, 1]$, u CCW smeru
- Koordinatu u dobijamo kao procenat obima koji je tačka P obišla od $z = 0$
- Najjednostavnije je izračunati $\theta = \arctan \frac{z}{x}$, a zatim $u = \frac{\theta}{2\pi}$, međutim $\arctan \frac{z}{x}$ daje vrednost $\theta \in (-\frac{\pi}{2}, \frac{\pi}{2})$ i preslikava dve dijametralno suprotne tačke na krugu u isti ugao θ
- Problem se rešava razmatranjem slučajeva:

$$\begin{aligned} \theta &= \arctan(z/x) \\ u &= \begin{cases} \frac{\theta}{2\pi} & \text{za } \theta < 0 \quad [0.0, 0.5] \\ 1 - \frac{\theta}{2\pi} & \text{za } \theta \geq 0 \quad [0.5, 1] \end{cases} \\ v &= y \end{aligned}$$



Preslikavanje teksture na sferu

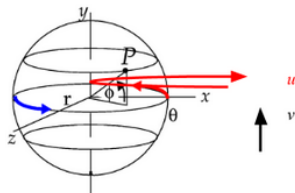
- Koordinatu u dobijamo isto kao kod valjka: kao procenat obima koji je tačka P obišla od $z = 0$
- Koordinatu v dobijamo kao funkciju geografske širine ϕ : računamo ugao ϕ koji tačka P zahvata sa ravni Oxz
- Na polovima, za $v = 0$ i $v = 1$ javljaju se singularnosti i u se postavlja na neku predefinisanu vrednost, npr. $u = 0.5$

$$\theta = \arctan \frac{z}{x}$$

$$u = \begin{cases} \frac{\theta}{2\pi} \text{ za } \theta < 0 \\ 1 - \frac{\theta}{2\pi} \text{ za } \theta \geq 0 \end{cases}$$

$$\phi = \arcsin \frac{y}{r}, \quad -\frac{\pi}{2} \leq \phi \leq \frac{\pi}{2}$$

$$v = \frac{\phi}{\pi} + \frac{1}{2}$$



Preslikavanje tekstura u slučaju proizvoljne mreže trouglova

- U opštem slučaju ovakva preslikavanja nije lako odrediti

Preslikavanje tekstura u slučaju proizvoljne mreže trouglova

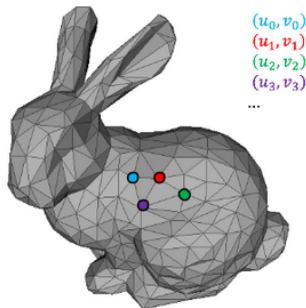
- U opštem slučaju ovakva preslikavanja nije lako odrediti
- Za proizvoljnu mrežu trouglova, nemamo pristup nekoj finoj funkciji $g(u, v) = (x, y, z)$ koja definiše oblik

Preslikavanje tekstura u slučaju proizvoljne mreže trouglova

- U opštem slučaju ovakva preslikavanja nije lako odrediti
- Za proizvoljnu mrežu trouglova, nemamo pristup nekoj finoj funkciji $g(u, v) = (x, y, z)$ koja definiše oblik
- Umesto toga, cilj je definisati (u, v) koordinate za svako teme mreže

Preslikavanje tekstura u slučaju proizvoljne mreže trouglova

- U opštem slučaju ovakva preslikavanja nije lako odrediti
- Za proizvoljnu mrežu trouglova, nemamo pristup nekoj finoj funkciji $g(u, v) = (x, y, z)$ koja definiše oblik
- Umesto toga, cilj je definisati (u, v) koordinate za svako teme mreže
- Videćemo uskoro da ako to znamo, lako je odrediti (u, v) koordinate za svaku tačku mreže

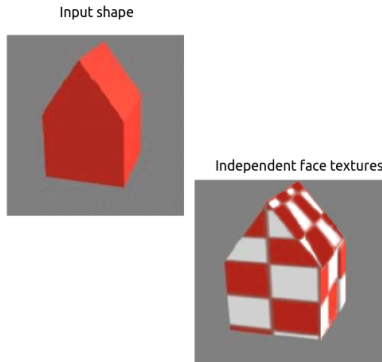


Preslikavanje složenih geometrijskih tela (1. način)

- Svaku poligonalnu stranu možemo nezavisno teksturisati

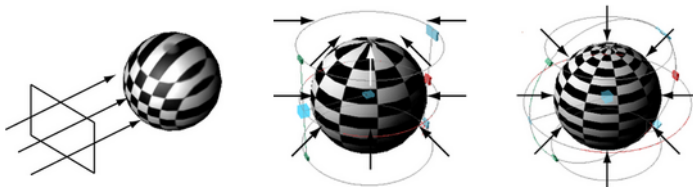
Preslikavanje složenih geometrijskih tela (1. način)

- Svaku poligonalnu stranu možemo nezavisno teksturisati
- Ovakav vid preslikavanja može rezultovati ružnim spojevima tamo gde se strane susreću



Preslikavanje složenih geometrijskih tela (2. način)

- Ukoliko je objekat približno ravanski, cilindrični ili sferni možemo koristiti formule za parametrizaciju ovih primitivnih oblika

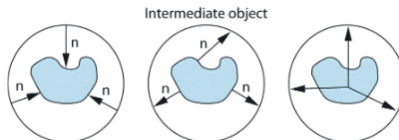


Preslikavanje složenih geometrijskih tela (2. način)

- Oko objekta se opisuje granični opseg



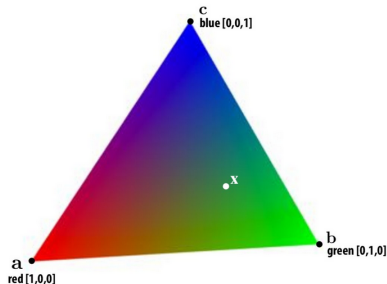
- Izvodimo **dvostepeno preslikavanje** u dva koraka:
 - vrši se preslikavanje teksture na granični opseg
 - vrši se preslikavanje sa graničnog opsega na ciljni objekat
 - određuje se presek normala graničnog opsega sa ciljnom površi
 - određuje se presek normala ciljne površi sa graničnim opsegom
 - određuje se presek vektora iz centra ciljne površi sa graničnim opsegom



Ručno preslikavanje tekstura

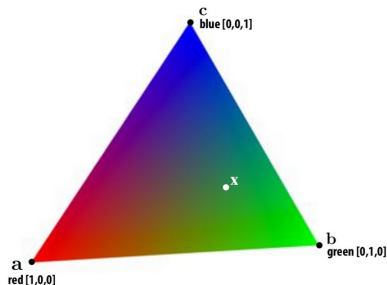
- Objekti sa kojima se radi u video-igrama i na filmu su, po pravilu, složeniji od prethodno razmatranih
- Potrebna nam je precizna kontrola kako slika teksture izgleda na objektu
- Iz tog razloga se za preslikavanje tekstura koriste programi za 3D modelovanje kao Blender, Maya,...

Uzorkovanje boje u tački trougla



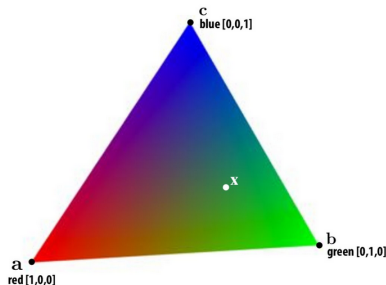
- Kako odrediti boju u tački x u unutrašnjosti trougla?

Uzorkovanje boje u tački trougla



- Kako odrediti boju u tački x u unutrašnjosti trougla?
- Potrebno je izvršiti interpolaciju vrednosti boje u temenima

Uzorkovanje boje u tački trougla



- Kako odrediti boju u tački x u unutrašnjosti trougla?
- Potrebno je izvršiti interpolaciju vrednosti boje u temenima
- Interpolacija u trouglu se vrši korišćenjem **baricentričnih koordinata**
 - najpre definišemo tačku trougla u terminima njegova tri temena
 - koristimo ovu reprezentaciju da izračunamo boju te tačke

Linearna interpolacija u 1D

- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2

Linearna interpolacija u 1D

- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2
- Proizvoljnu tačku Q duži A_1A_2 možemo parametarski predstaviti kao $Q = (1 - t)A_1 + tA_2$, gde je $0 \leq t \leq 1$

Linearna interpolacija u 1D

- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2
- Proizvoljnu tačku Q duži A_1A_2 možemo parametarski predstaviti kao $Q = (1 - t)A_1 + tA_2$, gde je $0 \leq t \leq 1$
- Drugačije rečeno važi: $Q = t_1A_1 + t_2A_2$, $t_1 + t_2 = 1$, $t_1, t_2 \geq 0$

Linearna interpolacija u 1D

- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2
- Proizvoljnu tačku Q duži A_1A_2 možemo parametarski predstaviti kao $Q = (1 - t)A_1 + tA_2$, gde je $0 \leq t \leq 1$
- Drugačije rečeno važi: $Q = t_1A_1 + t_2A_2$, $t_1 + t_2 = 1$, $t_1, t_2 \geq 0$
- t_1 i t_2 su **baricentrične koordinate duži** A_1A_2

Linearna interpolacija u 1D

- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2
- Proizvoljnu tačku Q duži A_1A_2 možemo parametarski predstaviti kao $Q = (1 - t)A_1 + tA_2$, gde je $0 \leq t \leq 1$
- Drugačije rečeno važi: $Q = t_1A_1 + t_2A_2$, $t_1 + t_2 = 1$, $t_1, t_2 \geq 0$
- t_1 i t_2 su **baricentrične koordinate duži** A_1A_2
- Duž je konveksna linearna kombinacija njenih krajnjih tačaka

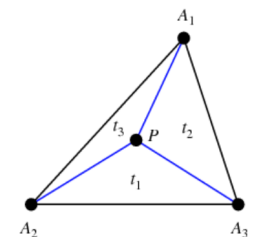
Linearna interpolacija u 1D

- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2
- Proizvoljnu tačku Q duži A_1A_2 možemo parametarski predstaviti kao $Q = (1 - t)A_1 + tA_2$, gde je $0 \leq t \leq 1$
- Drugačije rečeno važi: $Q = t_1A_1 + t_2A_2$, $t_1 + t_2 = 1$, $t_1, t_2 \geq 0$
- t_1 i t_2 su **baricentrične koordinate duži** A_1A_2
- Duž je konveksna linearna kombinacija njenih krajnjih tačaka
- Parametre t_1 i t_2 možemo videti kao težine, pa je duž težinski prosek krajnjih tačaka duži

Linearna interpolacija u 1D

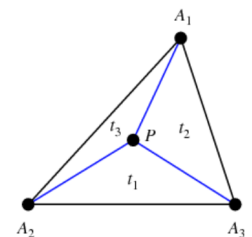
- Neka je data vrednost neke funkcije f u tačkama A_1 i A_2 i želimo da nađemo vrednost funkcije f u proizvoljnoj tački Q duži A_1A_2
- Proizvoljnu tačku Q duži A_1A_2 možemo parametarski predstaviti kao $Q = (1 - t)A_1 + tA_2$, gde je $0 \leq t \leq 1$
- Drugačije rečeno važi: $Q = t_1A_1 + t_2A_2$, $t_1 + t_2 = 1$, $t_1, t_2 \geq 0$
- t_1 i t_2 su **baricentrične koordinate duži** A_1A_2
- Duž je konveksna linearna kombinacija njenih krajnjih tačaka
- Parametre t_1 i t_2 možemo videti kao težine, pa je duž težinski prosek krajnjih tačaka duži
- Vrednost proizvoljnog svojstva koje se linearno menja duž duži A_1A_2 može se izraziti kao
$$f(Q) = t_1 \cdot f(A_1) + t_2 \cdot f(A_2), \quad t_1 + t_2 = 1, \quad t_1, t_2 \geq 0$$

Cilj baricentričnih koordinata



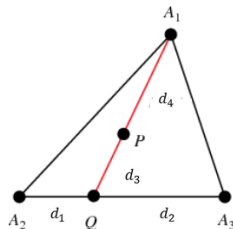
- Baricentrične koordinate se mogu uopštiti na trougao
- Neka je dat $\triangle A_1A_2A_3$ i proizvoljna tačka P trougla $A_1A_2A_3$
- Koje težine treba staviti na svako od temena trougla tako da trougao bude savršeno izbalansiran u datoj tački?

Cilj baricentričnih koordinata



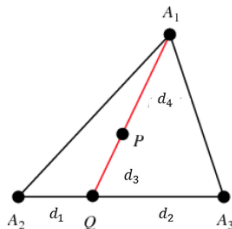
- Baricentrične koordinate se mogu uopštiti na trougao
- Neka je dat $\triangle A_1A_2A_3$ i proizvoljna tačka P trougla $A_1A_2A_3$
- Koje težine treba staviti na svako od temena trougla tako da trougao bude savršeno izbalansiran u datoj tački?
- Što je tačka P bliža temenu A_1 uticaj tačke A_1 je veći, a što je udaljenija od temena A_1 njen doprinos je manji

Linearna interpolacija u 2D



- Neka je Q presek prave A_1P i prave A_2A_3
- Zapišimo tačku Q kao težinsku sumu tačaka A_2 i A_3 , a tačku P kao težinsku sumu tačaka Q i A_1

Linearna interpolacija u 2D



- Neka je Q presek prave A_1P i prave A_2A_3
- Zapišimo tačku Q kao težinsku sumu tačaka A_2 i A_3 , a tačku P kao težinsku sumu tačaka Q i A_1

$$Q = (1 - t)A_2 + tA_3, \quad t = d_1/(d_1 + d_2)$$

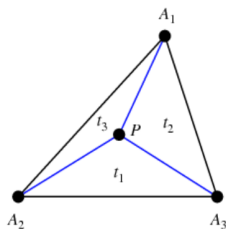
$$P = (1 - s)Q + sA_1, \quad s = d_3/(d_3 + d_4)$$

- Zamenom prve jednačine u drugu dobijamo

$$P = sA_1 + (1 - s)(1 - t)A_2 + (1 - s)tA_3$$

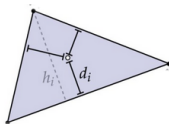
- Koeficijenti uz A_1 , A_2 i A_3 su nenegativni i njihova suma jednaka je 1

Linearna interpolacija u 2D



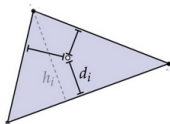
- Tačka P trougla $A_1A_2A_3$ može se predstaviti kao $t_1A_1 + t_2A_2 + t_3A_3$, gde je $t_1 + t_2 + t_3 = 1$ i $t_1, t_2, t_3 \geq 0$
- Brojeve t_1, t_2 i t_3 nazivamo **baricentričnim koordinatama** tačke P u odnosu na $\triangle A_1A_2A_3$
- Teksturne koordinate u i v tačke P možemo na ovaj način odrediti na osnovu u i v koordinata tačaka A_1, A_2 i A_3

Interpretacija baricentričnih koordinata



- Baricentrične koordinate tačke P možemo da tumačimo i putem rastojanja tačke od stranica trougla:
 - težina uz teme A_1 treba da bude proporcionalna rastojanju tačke P do stranice A_2A_3

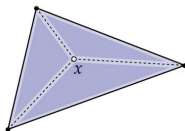
Interpretacija baricentričnih koordinata



- Baricentrične koordinate tačke P možemo da tumačimo i putem **rastojanja tačke od stranica trougla**:
 - težina uz teme A_1 treba da bude proporcionalna rastojanju tačke P do stranice A_2A_3
- Baricentrične koordinate tačke P dobijamo tako što rastojanja tačke P do stranica A_1A_2 , A_2A_3 i A_3A_1 podelimo dužinama visina trouglova iz temena A_3 , A_1 i A_2 redom:

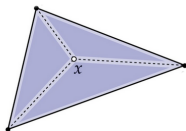
$$t_i = \frac{d_i}{h_i}$$

Interpretacija baricentričnih koordinata



- Baricentrične koordinate tačke P možemo da tumačimo i putem površina trouglova:
 - težina uz teme A_1 treba da bude proporcionalna površini $\triangle PA_2A_3$

Interpretacija baricentričnih koordinata



- Baricentrične koordinate tačke P možemo da tumačimo i putem površina trouglova:
 - težina uz teme A_1 treba da bude proporcionalna površini $\triangle PA_2A_3$
- Baricentrične koordinate tačke P dobijamo tako što površine $\triangle PA_1A_2$, $\triangle PA_2A_3$ i $\triangle PA_3A_1$ podelimo površinom $\triangle A_1A_2A_3$:

$$t_3 = \frac{p(\triangle PA_1A_2)}{p(\triangle A_1A_2A_3)}$$

Perspektivno nekorektna interpolacija

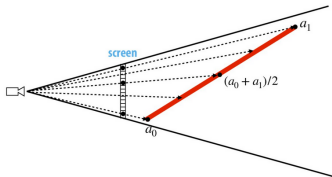
- Neka je cilj interpolirati vrednost funkcije f nad trouglom ABC u 3D, pri čemu je poznato $f(A), f(B), f(C)$
- Neka je $A'B'C'$ perspektivna projekcija trougla ABC
- Želimo da odredimo vrednost funkcije f u tački P trougla $A'B'C'$

Perspektivno nekorektna interpolacija

- Neka je cilj interpolirati vrednost funkcije f nad trouglom ABC u 3D, pri čemu je poznato $f(A), f(B), f(C)$
- Neka je $A'B'C'$ perspektivna projekcija trougla ABC
- Želimo da odredimo vrednost funkcije f u tački P trougla $A'B'C'$
- Izračunamo baricentrične koordinate tačke P : $P = \alpha A' + \beta B' + \gamma C'$
- **Ne važi nužno:** $f(P) = \alpha f(A) + \beta f(B) + \gamma f(C)$

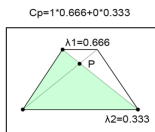
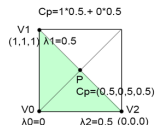
Perspektivno nekorektna interpolacija

- Neka je cilj interpolirati vrednost funkcije f nad trouglom ABC u 3D, pri čemu je poznato $f(A), f(B), f(C)$
- Neka je $A'B'C'$ perspektivna projekcija trougla ABC
- Želimo da odredimo vrednost funkcije f u tački P trougla $A'B'C'$
- Izračunamo baricentrične koordinate tačke P : $P = \alpha A' + \beta B' + \gamma C'$
- **Ne važi nužno:** $f(P) = \alpha f(A) + \beta f(B) + \gamma f(C)$
- Perspektivna projekcija nije afina transformacija te baricentrična interpolacija vrednosti na trouglu sa različitim dubinama ne daje dobre rezultate



Razlika u baricentričnim koordinatama u 3D prostoru objekta i u prostoru ekrana

- Neka je dat kvadrat u 3D prostoru koji nije paralelan ravni projekcije i neka se on projektuje u trapez prikazan na donjoj slici

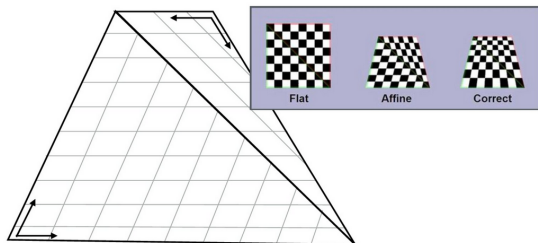


© www.scratchapixel.com

- Tačka P se slika u tačku P'
- U kvadratu važi: $P = 0.5 \cdot V_1 + 0.5 \cdot V_2$
- U trapezu važi: $P' = 0.666 \cdot V'_1 + 0.333 \cdot V'_2$

Primer: perspektivno nekorektna interpolacija

- Razmotrimo kvadrat u 3D prostoru koji se projektuje u četvorougao na slici
- Delimo ga na dva trougla koja treba teksturisati
- Ako preslikamo teksture korišćenjem baricentričnih koordinata za 2D (projektovane) koordinate, gubimo neprekidnost interpolacije na mestu gde je četvorougao izdijeljen na trouglove



Perspektivno korektna interpolacija

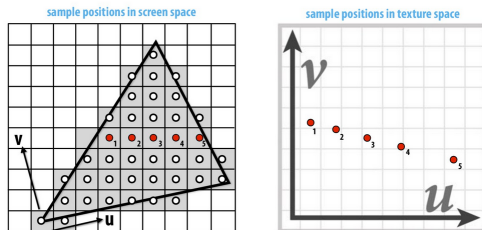
- Cilj: interpolirati vrednost funkcije f zadate u temenima 3D trougla
- Recept:
 - izračunati vrednost z koordinate za svako teme
 - izračunati vrednost $Z = 1/z$ i $g = f/z$ za svako teme
 - izvršiti interpolaciju vrednosti Z i g korišćenjem standardnih (2D) baricentričnih koordinata projekcija trougla
 - za svaki od fragmenata, podeliti interpolisanu vrednost funkcije g interpolisanom vrednošću funkcije Z da bismo dobili finalnu vrednost funkcije f fragmenta

Više detalja i izvođenja dostupni su na:

- <https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/visibility-problem-depth-buffer-depth-interpolation>
- <https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/perspective-correct-interpolation-vertex-attributes>

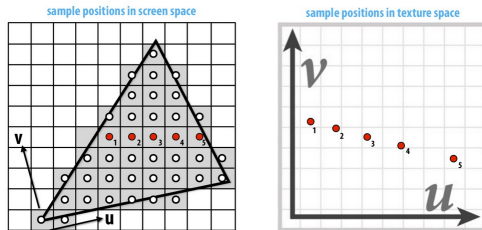
Vizualizacija uzoraka teksture

- Trouglovi se projektuju iz 3D prostora u ravan, pa će pikseli u prostoru ekrana odgovarati regionima promenljive veličine i lokacije na teksturi
- Lokacije u kojima vršimo uzorkovanje **u prostoru ekrana** imaju ravnomernu raspodelu
- Međutim, lokacije u kojima vršimo uzorkovanje **u prostoru tekstura** nemaju ravnomernu raspodelu



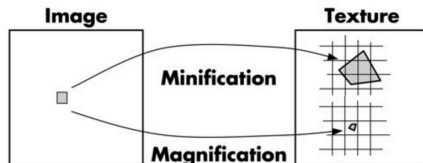
Vizualizacija uzoraka teksture

- Trouglovi se projektuju iz 3D prostora u ravan, pa će pikseli u prostoru ekrana odgovarati regionima promenljive veličine i lokacije na teksturi
- Lokacije u kojima vršimo uzorkovanje **u prostoru ekrana** imaju ravnomernu raspodelu
- Međutim, lokacije u kojima vršimo uzorkovanje **u prostoru tekstura** nemaju ravnomernu raspodelu



- Nepravilan obrazac uzorkovanja rezultuje **teksturnim alijasingom**

Uvećanje vs. umanjenje teksture



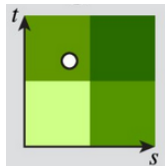
- **Uvećanje teksture** (magnifikacija) – jednostavnije:
 - npr. kada je objekat jako blizu kameri
 - pojedinačni piksel ekrana se preslikava u veoma mali region teksture
 - interpolacijom vrednosti u centrima susednih piksela može se dobiti vrednost u necelobrojnoj tački
- **Umanjenje teksture** (minifikacija) – teže:
 - npr. kada je objekat jako daleko od kamere
 - pojedinačni piksel ekrana se preslikava u veliki region teksture
 - potrebno je izračunati prosečnu vrednost dela teksture pokrivenog pikselom da bismo izbegli aliasing efekat

Uvećanje teksture (1. pristup)

- Píksel ekrana se preslikava u mali region teksture
- Kako naći vrednost teksture u necelobrojnoj lokaciji (u, v) ?

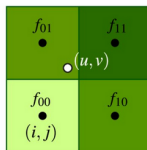
Uvećanje teksture (1. pristup)

- Píksel ekrana se preslikava u mali region teksture
- Kako naći vrednost teksture u necelobrojnoj lokaciji (u, v) ?
- Prvi pristup: **metoda najbližeg suseda** – pokupimo vrednost najbližeg piksela teksture
- Efikasno, ali ružno



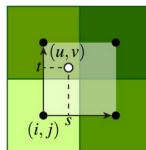
Uvećanje tekstone (2. pristup)

- Drugi pristup: **bilinearna interpolacija** – najpre vršimo linearnu interpolaciju po horizontali, pa po vertikali



$$i = \lfloor u - \frac{1}{2} \rfloor$$

$$j = \lfloor v - \frac{1}{2} \rfloor$$



$$s = u - (i + \frac{1}{2}) \in [0, 1]$$

$$t = v - (j + \frac{1}{2}) \in [0, 1]$$

linear (each row)



$$(1-s)f_{01} + sf_{11}$$

$$(1-s)f_{00} + sf_{10}$$

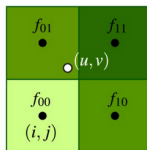
bilinear



$$(1-t)((1-s)f_{00} + sf_{10}) + t((1-s)f_{01} + sf_{11})$$

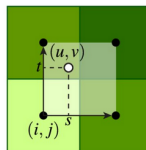
Uvećanje tekstone (2. pristup)

- Drugi pristup: **bilinearna interpolacija** – najpre vršimo linearnu interpolaciju po horizontali, pa po vertikali



$$i = \lfloor u - \frac{1}{2} \rfloor$$

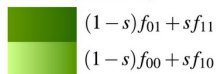
$$j = \lfloor v - \frac{1}{2} \rfloor$$



$$s = u - (i + \frac{1}{2}) \in [0, 1]$$

$$t = v - (j + \frac{1}{2}) \in [0, 1]$$

linear (each row)



$$(1-s)f_{01} + sf_{11}$$

$$(1-s)f_{00} + sf_{10}$$

bilinear



$$(1-t)((1-s)f_{00} + sf_{10}) + t((1-s)f_{01} + sf_{11})$$

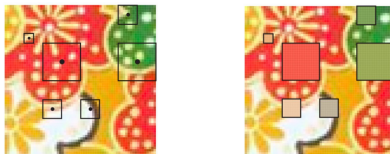
- Da li bismo dobili istu vrednost da smo prvo vršili interpolaciju po vertikali, pa po horizontali?

Umanjenje teksture

- Pikel ekrana se preslikava u veliki region teksture
- Ako bismo samo uzeli vrednost teksture u centru regiona, dobili bismo **teksturni aliasing** (boju koja se menja ako se uzorak vrlo malo pomeri)

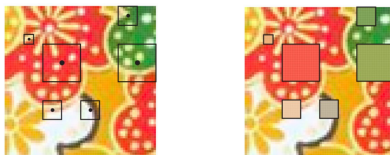
Umanjenje tekstone

- Píksel ekrana se preslikava u veliki region teksture
- Ako bismo samo uzeli vrednost teksture u centru regiona, dobili bismo **teksturni aliasing** (boju koja se menja ako se uzorak vrlo malo pomeri)
- Idealno bi bilo uzeti **prosečnu vrednost** regiona teksture, ali bi ovo bilo neefikasno



Umanjenje teksture

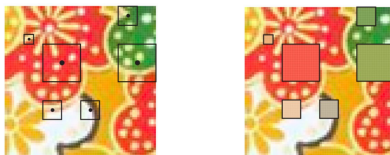
- Piskel ekrana se preslikava u veliki region teksture
- Ako bismo samo uzeli vrednost teksture u centru regiona, dobili bismo **teksturni aliasing** (boju koja se menja ako se uzorak vrlo malo pomeri)
- Idealno bi bilo uzeti **prosečnu vrednost** regiona teksture, ali bi ovo bilo neefikasno



- Umesto toga, možemo koristiti **prefiltriranje**: **unapred** izračunati proseke (jednom) i u vreme izvršavanja (veliki broj puta) koristiti dobijene vrednosti

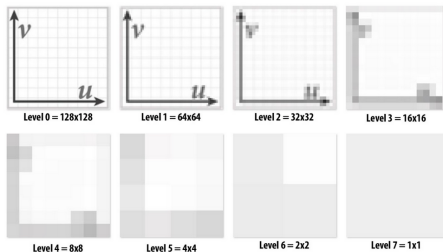
Umanjenje teksture

- Piksela ekrana se preslikava u veliki region teksture
- Ako bismo samo uzeli vrednost teksture u centru regiona, dobili bismo **teksturni aliasing** (boju koja se menja ako se uzorak vrlo malo pomeri)
- Idealno bi bilo uzeti **prosečnu vrednost** regiona teksture, ali bi ovo bilo neefikasno



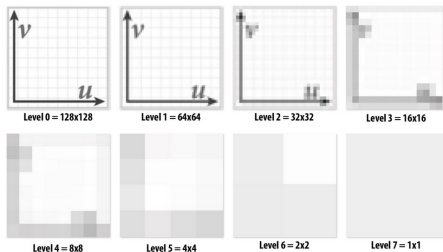
- Umesto toga, možemo koristiti **prefiltriranje**: unapred izračunati proseke (jednom) i u vreme izvršavanja (veliki broj puta) koristiti dobijene vrednosti
- Koje proseke treba čuvati? Ne možemo ih sve unapred izračunati
- Za objekte koji su blizu trebaju nam slike visoke rezolucije, a za udaljene objekte slike manje rezolucije

MIP mape



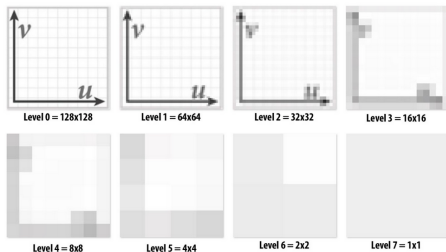
- **MIP mape** (lat. multum in parvo, eng. much in little)
- Osnovna ideja: sačuvati unapred smanjenu sliku u “svakoј mogućoj razmeri”

MIP mape



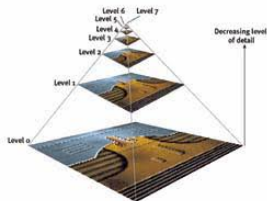
- **MIP mape** (lat. multum in parvo, eng. much in little)
- Osnovna ideja: sačuvati unapred smanjenu sliku u “svakoј mogućoj razmeri”
- Tekseli na višem nivou čuvaju proseke tekstura sa prethodnog nivoa
- Za udaljene objekte koristićemo teksture manje rezolucije

MIP mape



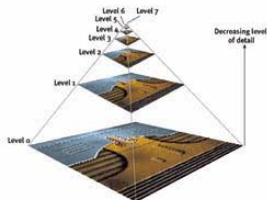
- **MIP mape** (lat. multum in parvo, eng. much in little)
- Osnovna ideja: sačuvati unapred smanjenu sliku u “svakoј mogućoj razmeri”
- Tekseli na višem nivou čuvaju proseke tekstura sa prethodnog nivoa
- Za udaljene objekte koristićemo teksture manje rezolucije
- Moguće je pogledati samo jedan piksel MIP mape odgovarajućeg nivoa
- Ovo je primer LOD (level of detail) principa u računarskoј grafici

MIP mape



- Koliki su troškovi skladištenja MIP mape?

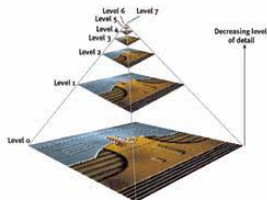
MIP mape



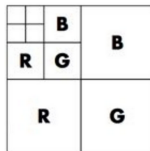
- Koliki su troškovi skladištenja MIP mape?

		B	B
R	G		
R		G	

MIP mape

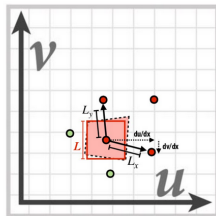
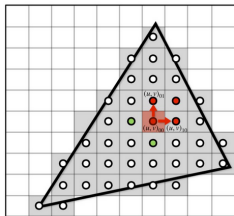


- Koliki su troškovi skladištenja MIP mape?



- $\sum_{i=1}^{\infty} \frac{1}{4^i} = \frac{1}{3}$

Odabir nivoa MIP mape



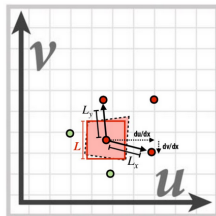
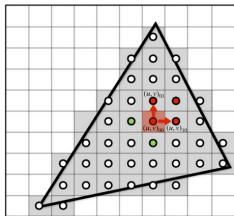
- Razmotrimo koliko se brzo menjaju vrednosti u i v koordinate duž x i y ose, računanjem razlike vrednosti teksturnih koordinata susednih piksela

$$\frac{du}{dx} = u_{10} - u_{00}, \quad \frac{dv}{dx} = v_{10} - v_{00}$$

$$\frac{du}{dy} = u_{01} - u_{00}, \quad \frac{dv}{dy} = v_{01} - v_{00}$$

$$L_x^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{dv}{dx}\right)^2 \quad L_y^2 = \left(\frac{du}{dy}\right)^2 + \left(\frac{dv}{dy}\right)^2$$

Odabir nivoa MIP mape



- Razmotrimo koliko se brzo menjaju vrednosti u i v koordinate duž x i y ose, računanjem razlike vrednosti teksturnih koordinata susednih piksela

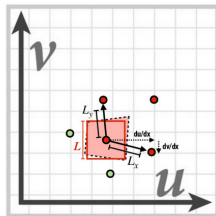
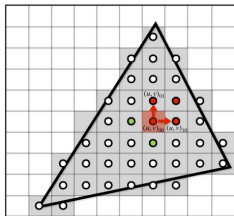
$$\frac{du}{dx} = u_{10} - u_{00}, \quad \frac{dv}{dx} = v_{10} - v_{00}$$

$$\frac{du}{dy} = u_{01} - u_{00}, \quad \frac{dv}{dy} = v_{01} - v_{00}$$

$$L_x^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{dv}{dx}\right)^2 \quad L_y^2 = \left(\frac{du}{dy}\right)^2 + \left(\frac{dv}{dy}\right)^2$$

- Veća od dužina vektora L_x i L_y nam daje neku procenu koliki region teksture pokriva jedan piksel: $L = \sqrt{\max(L_x^2, L_y^2)}$

Odabir nivoa MIP mape



- Razmotrimo koliko se brzo menjaju vrednosti u i v koordinate duž x i y ose, računanjem razlike vrednosti teksturnih koordinata susednih piksela

$$\frac{du}{dx} = u_{10} - u_{00}, \quad \frac{dv}{dx} = v_{10} - v_{00}$$

$$\frac{du}{dy} = u_{01} - u_{00}, \quad \frac{dv}{dy} = v_{01} - v_{00}$$

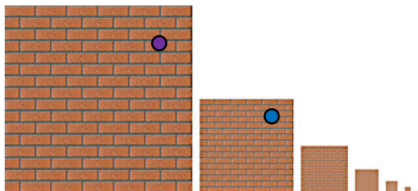
$$L_x^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{dv}{dx}\right)^2 \quad L_y^2 = \left(\frac{du}{dy}\right)^2 + \left(\frac{dv}{dy}\right)^2$$

- Veća od dužina vektora L_x i L_y nam daje neku procenu koliki region teksture pokriva jedan piksel: $L = \sqrt{\max(L_x^2, L_y^2)}$
- Pošto je kod MIP mape svaka naredna slika teksture duplo manje dimenzije, nivo d MIP mape koji treba koristiti za dati piksel dobijamo kao: $d = \log_2 L$

Problemi pri korišćenju MIP mapa

- Ako koristimo nivo MIP mape najbliži vrednosti d , možemo dobiti efekat da nivo detaljnosti prikaza naglo “skoči”
- Umesto da biramo tačno jedan nivo MIP mape koji odgovara najbližem celom broju, možemo da koristimo baš vrednost d i da vršimo interpolaciju između dva susedna nivoa MIP mape
- Ovde dolazimo do tzv. **trilinearne interpolacije**: mešamo teksture sa dva susedna nivoa MIP mape

$$(1 - \alpha) \text{●} + \alpha \text{●}$$



Anizotropno filtriranje

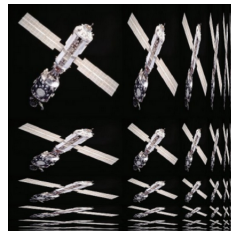
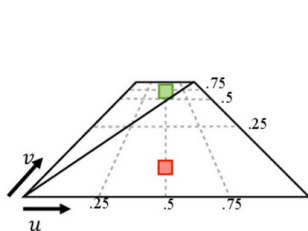
- Trilinearno filtriranje pretpostavlja da se uzorci smanjuju istom brzinom duž u i v koordinate

Anizotropno filtriranje

- Trilinearno filtriranje pretpostavlja da se uzorci smanjuju istom brzinom duž u i v koordinate
- Bilinearno i trilinearno filtriranje su primeri tzv. **izotropnog filtriranja**

Anizotropno filtriranje

- Trilinearno filtriranje pretpostavlja da se uzorci smanjuju istom brzinom duž u i v koordinate
- Bilinearno i trilinearno filtriranje su primeri tzv. **izotropnog filtriranja**
- Razmotrimo primer ravni koja je zarotirana u odnosu na kameru – promene duž koordinate v su značajno veće nego duž u



- Kod **anizotropnog filtriranja** se pravi nova mapa teksture koja nezavisno smanjuje sliku po x i y osi: $(d_x, d_y) = (\log_2 \sqrt{L_x^2}, \log_2 \sqrt{L_y^2})$

Druge vrste preslikavanja

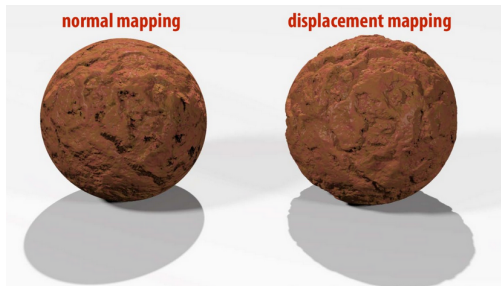
- Ideja preslikavanja kodiranog slikom može se preneti mnogo šire nego samo za dobijanje boje površi
- Mnogi atributi u procesu renderovanja mogu se kodirati slikom:
 - razne osobine materijala: sjajnost, izgribanost,...
 - normale površi, transparentnost, refleksivnost,...
 - boja svetlosti, intenzitet svetlosti,...
 - pozicija objekta

Druge vrste preslikavanja

- Ideja preslikavanja kodiranog slikom može se preneti mnogo šire nego samo za dobijanje boje površi
- Mnogi atributi u procesu renderovanja mogu se kodirati slikom:
 - razne osobine materijala: sjajnost, izgribanost,...
 - normale površi, transparentnost, refleksivnost,...
 - boja svetlosti, intenzitet svetlosti,...
 - pozicija objekta
- Teksture mogu sadržati različite vrste informacija
- Može se koristiti i kombinacija većeg broja mapa

Druge vrste preslikavanja

- **Preslikavanje normala** (normal mapping) – vektori normala se čuvaju u mapi normala, omogućavajući preciznije simulacije osvetljenja, geometrija objekta se ne menja
- **Preslikavanje pomeraja** (displacement mapping) – temena mreže poligona se pomeraju duž vektora normale za vrednost koja se čuva u mapi visine



Druge vrste preslikavanja

- **Preslikavanje neravnina** (bump mapping) – omogućava izmenu vektora normala tokom renderovanja na osnovu mape neravnina
- **Preslikavanje okruženja** (environment mapping) – koristi sliku okruženja za preslikavanje tekstura, omogućava simulaciju površi koje imaju visoku refleksiju

